

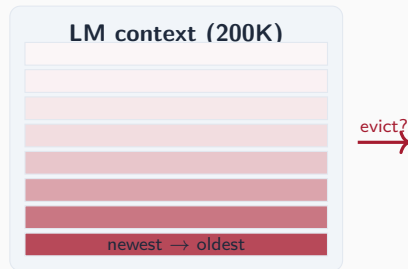
Cache-Informed Prompting for Recursive Language Models

Fucheng Warren Zhu

CS2640 · Spring 2026

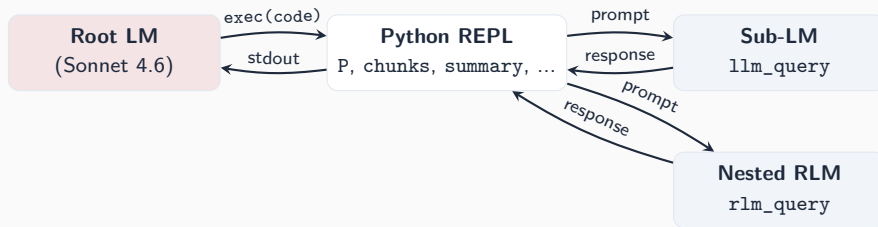
Context is a cache

- LLMs come with a 200K-token cache called *context*.
- When it fills, we need a policy for eviction.
- Popular policies are not very principled:
 - drop-oldest
 - summarize-and-collapse
 - persist to filesystem, keep an index in cache
- This course covered better answers: LRU, LFU, S3-FIFO, SIEVE, admission control.
- **Question.** Can we just tell the model to use a real policy?



Recursive Language Models

Zhang, Kraska & Khattab, arXiv:2512.24601 (2026)



- User prompt lives in the REPL as variable `P` — not in the context window.
- Root LM sees only metadata (length, prefix) and writes code to inspect / slice it.
- Variables persist; `del` evicts. The LM runs until it emits `FINAL(answer)`.

Paper results — RLM lifts every benchmark

Zhang+Kraska+Khattab 2026, Table 1 (GPT-5 row). Accuracy (%).

Method	CodeQA	BrowseComp+ (1K)	OOLONG	OOLONG-Pairs
<i>Task length</i>	<i>23K–4.2M</i>	<i>6M–11M</i>	<i>131K</i>	<i>32K</i>
Base Model	24.0*	0.0*	44.0	0.1
CodeAct + BM25	22.0*	51.0	38.0	24.7
Summary agent	58.0	70.5	46.0	0.1
RLM	62.0	91.3	56.5	58.0
RLM (no sub-calls)	58.0	88.0	36.0	43.9

* = method (sometimes) ran into input context limits.

- RLM scaffold lifts every benchmark over every other method.
- Biggest lift: OOLONG-Pairs (0.1% → 58%).
- *Recursion* matters: disabling sub-calls drops OOLONG-Pairs 58 → 43.9.
- **Cost-comparable to base model.** RLM pays for what it *reads*, not what's *available* — on long inputs you only need a small fraction. Risk: confused trajectories can recurse a long way before terminating, so cost has high variance.

Experimental design

Five benchmarks (same set as the paper):

- **S-NIAH** — 2K–262K filler with one injected key. Retrieval.
- **LongBench-v2 CodeQA** — 23K–4.2M tokens of code, 4-way MC.
- **BrowseComp+ (1K)** — 6M–11M tokens, multi-hop over 1K docs.
- **OOLONG** (trec_coarse) — 131K tokens, semantic aggregation.
- **OOLONG-Pairs** — 32K tokens, quadratic pairwise reasoning.

Eight methods:

- **Baselines (4):** `direct`, `summary_agent`, `codeact_bm25`, and `rlm_a0` (vanilla RLM, the RLM baseline).
- **Treatment arms (4):** a single cache-hint block appended to the RLM system prompt — see next slide.

Sonnet 4.6, via the Agent SDK over a Max-subscription OAuth token.

The four treatment arms

Each arm appends a single hint block to the upstream RLM system prompt.

Arm	Policy	What it tells the model
A1	LRU	"When you must evict, prefer to del the variable you have not touched recently." Tests temporal locality.
A2	S3-FIFO	"New variables are on probation. Promote to a main set only on re-reference; evict the rest. Track recently evicted in a ghost set." Tests one-hit-wonder filtering.
A3	SIEVE	"Walk variables in creation order. On each pass, evict the first one not re-referenced since last pass." Tests lazy promotion.
A4	theory primer	A short primer on <i>retention</i> vs. <i>reconstruction</i> cost, the LRU/LFU/S3-FIFO/SIEVE menu, and admission filtering. This is a non-prescriptive decision framework.

The control is `rlm_a0` (vanilla RLM with empty suffix). **What we want to learn:** does any of A1–A3 (directive) outperform `rlm_a0`? Does A4 (framework) behave differently from the directive arms?

Preliminary results — S-NIAH and CodeQA

We first ran on shorter context tasks as a preliminary investigation.

Method	S-NIAH (retrieval, 2K–262K)	CodeQA (reasoning, ~100K)
direct	1.00	1.00
summary_agent	1.00	0.33
codeact + BM25	0.43	0.33
rlm_a0 (vanilla RLM)	1.00	0.50
A1 (LRU)	0.67	0.00
A2 (S3-FIFO)	0.20	0.67
A3 (SIEVE)	0.20	—
A4 (theory primer)	0.60	1.00

Observations:

- Directive arms (A2, A3) collapse on retrieval — $1.00 \rightarrow 0.20$.
- Even the gentler primer (A4) drops to 0.60 at higher tiers.
- On reasoning the directive arms *help*: A2 lifts CodeQA $0.50 \rightarrow 0.67$.
- Vanilla RLM is the strongest cache-aware method on retrieval; no treatment arm beats it.

Findings and next steps

Preliminary results — we need much more experimentation on top to validate.

- (a) **Prompt hints can manufacture failures.** On S-NIAH directive arms led the model to evict its own answer. 1.00 $\rightarrow$ 0.20.
- (b) **Same hints, opposite signs across tasks.** The arm that hurt S-NIAH lifts CodeQA 0.50 $\rightarrow$ 0.67. Hints act as a prior — helps when it matches the task, hurts when it doesn't.
- (c) **The decision space is enormous.** A few hundred prompt tokens may not encode the right behavior across all task structures. The right layer might be the runtime, not the prompt.
- (d) **Efficiency is missing.** We tracked accuracy only. Open question: **how can eviction policies be KV-cache aware?** Policies that preserve the prefix keep KV reuse cheap.